

Aggregation in Relational Algebra ($\mathcal{G}$)

The $\mathcal{G}$ operator groups tuples and applies aggregate functions, analogous to SQL's `GROUP BY + HAVING`.

Syntax: $\underbrace{\text{grouping attrs}}_{\text{(optional)}} \mathcal{G} \underbrace{\text{agg functions}}_{\text{e.g. AVG(gpa)}}(R)$ Omit grouping attrs $\Rightarrow$ whole relation is one group.

Function	Meaning / NULL handling
COUNT(*)	Number of tuples. NULLs <i>counted</i> .
COUNT(A)	Non-NULL values in column A.
SUM / AVG(A)	Sum / average of A; NULLs <i>ignored</i> .
MIN / MAX(A)	Smallest / largest value; NULLs ignored.

Key rules. Grouping attributes appear unchanged in the result schema. An empty group yields COUNT = 0; all other aggregates yield NULL. Aggregate results need new attribute names.

RA expression	SQL equivalent
$\mathcal{G}_{\text{AVG(gpa)}}(Students)$	SELECT AVG(gpa) FROM Students
major $\mathcal{G}_{\text{AVG(gpa)}, \text{COUNT(*)}}(Students)$	SELECT major, AVG(gpa), COUNT(*) FROM Students GROUP BY major
$\sigma_{\text{AVG(gpa)} > 3.5}(\text{major } \mathcal{G}_{\text{AVG(gpa)}}(Students))$	... GROUP BY major HAVING AVG(gpa) > 3.5

Apply σ **after** $\mathcal{G}$ to filter groups. This is what **HAVING** does in SQL.

Views

A **view** is a named query that behaves like a table. There is no need to create a new base table when not everyone should see all columns, or when a complex join is reused often.

Type	How it works
Virtual (default)	Not stored. Query is re-run every time the view is accessed.
Materialized	Result is stored; must be refreshed after base-table changes.

Declare: CREATE [MATERIALIZED] VIEW *name* AS *query*;

Drop: DROP VIEW *name*;

Query: use the view name exactly like a base table in any SELECT.

Updates on views. Reads always work (the DB translates them to a SELECT on base tables). Writes (INSERT/UPDATE/DELETE) are only unambiguous when every column traces back to exactly one row in exactly one base table. Most views do not meet this bar, so writes are blocked by default.

```
-- Expose only safe columns to front-desk staff
CREATE VIEW PublicEmployee AS
  SELECT emp_id, name, phone, dept
  FROM Employee;           -- salary, ssn, rating hidden

-- Query the view exactly like a table
SELECT name FROM PublicEmployee WHERE dept = 'Engineering';
```

Triggers

A trigger is a stored procedure the database runs *automatically* in response to an event; you never call it yourself.

Clause	Meaning
BEFORE / AFTER / INSTEAD OF	When the trigger fires relative to the event.
INSERT / UPDATE / DELETE	The event that fires the trigger.
FOR EACH ROW	Body runs once per affected row.
FOR EACH STATEMENT	Body runs once per SQL statement (default).
REFERENCING NEW ROW AS n	Alias for the newly inserted/updated row.
REFERENCING OLD ROW AS o	Alias for the pre-update/delete row.

Use *INSTEAD OF* on a view to intercept inserts and route them to the correct base tables.

```
CREATE TRIGGER LogEnrollment
AFTER INSERT ON Took
REFERENCING NEW ROW AS n
FOR EACH ROW BEGIN
    INSERT INTO AuditLog VALUES(n.sid, 'enrolled', CURRENT_TIMESTAMP);
END;
```

Use Side A as a reference. Write actual SQL or relational algebra – not pseudocode. Boxes marked (*discuss*) are debriefed with the class after individual work. Schema for RA questions: *Students*(sid, name, major, gpa) from prior lectures.

Box 1 RA Aggregation (8 min, discuss)

(a) Write the RA expression for: the number of students in each major. Label the count attribute `num_students`.

(b) Extend your expression from (a) to keep only majors with **more than one** student. Which RA operator do you apply after $\mathcal{G}$, and why?

(c) Write the SQL equivalent for (b).

(d) A classmate writes `WHERE COUNT(*) > 1` instead of `HAVING COUNT(*) > 1`. In one sentence, why does this fail?

.....

Box 2 Views (6 min)

(a) Write a `CREATE VIEW` statement called `HighGPA` that contains the `sid` and `name` of every student whose GPA is above 3.5.

(b) Write a query that uses `HighGPA` to return the names of high-GPA students in the `CS` major.

(c) Your colleague tries to run `INSERT INTO HighGPA VALUES(999, 'Zara')`. Will this succeed? In one sentence, explain why or why not.

.....
.....

Box 3 Triggers (8 min, discuss)

Use the table `Enrolled(sid, cid, grade)` and `GradeLog(sid, cid, old_grade, new_grade, changed_at)`.

(a) Write a trigger `TrackGradeChange` that fires **after** an update on `Enrolled` and inserts a row into `GradeLog` recording the old and new grade.

(b) Should this trigger use `FOR EACH ROW` or `FOR EACH STATEMENT`? Circle one and explain in one sentence.

.....

(c) If a single `UPDATE` statement changes grades for 50 students at once, how many times does a `FOR EACH ROW` trigger fire vs. a `FOR EACH STATEMENT` trigger?

.....